4 Τελεστές και βασική Είσοδος και Έξοδος

Η Java ενσωματώνει μια σειρά από συναρτήσεις που χρησιμεύουν ώστε να μπορούμε να κάνουμε διάφορες πράξεις (operations). Κάθε πράξη περιλαμβάνει τον τελεστή (operator) και τους τελεστέους (operands). Για παράδειγμα, στην πράξη 5+3, τελεστής είναι το σύμβολο της πρόσθεσης, + και τελεστέοι τα 5 και 3. Προκειμένου να διευκολυνθεί η έκφραση στο πρόγραμμα τέτοιων πράξεων, η Java παρέχει ειδικό συντακτικό κλήσης των τελεστών έτσι ώστε οι ανάλογες εκφράσεις να μην ακολουθούν το τυπικό συντακτικό κλήσης συναρτήσεων αλλά να διατυπώνονται κατά τρόπο που ομοιάζει με την εκτεταμένη χρήση τους στα μαθηματικά και άλλους τομείς.

Ένα άλλο ζήτημα, αναγκαίο για τα προγράμματά μας, είναι η βασική είσοδος και έξοδος. Πρόκειται για τις διαδικασίες που επιτρέπουν την επικοινωνία και αλληλεπίδραση μεταξύ χρήστη και προγράμματος. Πώς δηλαδή θα πάρω δεδομένα από τον χρήστη και πώς θα παρουσιάσω σε αυτόν τις πληροφορίες που θέλω;

Σε αυτό το κεφάλαιο παρουσιάζουμε τους βασικούς τελεστές που υποστηρίζει η Java [1] ταξινομώντας τους σε κατηγορίες ανάλογα με τα χαρακτηριστικά τους καθώς και τις βασικές λειτουργίες εισόδου από το πληκτρολόγιο και εξόδου στην οθόνη.

4.1 Τελεστές

Οι τελεστές (operators) στην Java οργανώνονται ανάλογα με τον αριθμό τελεστέων (operands) που δέχονται. Έτσι, έχουμε τους μοναδιαίους (unary) τελεστές, δηλαδή αυτούς που δέχονται έναν τελεστέο, τους δυαδικούς (binary), δηλαδή αυτούς με δύο τελεστέους και έναν τριαδικό (ternary) τελεστή. Επίσης, οι τελεστές κατηγοριοποιούνται σε αριθμητικούς (arithmetic), υποθετικούς (conditional) ή λογικούς (logical), σχεσιακούς (relational), τελεστές ψηφίου (bitwise) και τους τελεστές εκχώρησης (assignment).
Σε μια αλγεβρική έκφραση είναι δυνατό να συνδυάζονται περισσότεροι από έναν τελεστές. Για παράδειγμα, στην αλγεβρική έκφραση 2+3x5, συνδυάζονται οι τελεστές πρόσθεσης και πολλαπλασιασμού. Όπως γνωρίζουμε από την άλγεβρα, ο πολλαπλασιασμός έχει προτεραιότητα σε σχέση με την πρόσθεση. Επομένως, 2+3x5=17. Παρόμοια και στις εκφράσεις (expressions) της Java είναι δυνατό να συνδυάζονται πολλοί τελεστές. Επομένως, για τον συνεπή υπολογισμό των εκφράσεων έχουν καθοριστεί προτεραιότητες και για τους τελεστές της Java. Στον πίνακα 4.1 παρουσιάζονται οι τελεστές διατεταγμένοι από υψηλότερη σε χαμηλότερη προτεραιότητα. Τελεστές στην ίδια γραμμή του πίνακα 4.1 έχουν ίση προτεραιότητα.

Τύπος	**Λειτουργία**	**Προτεραιότητα**
Μοναδιαίος	Μεταθεματική αύξηση/μείωση	++, --
	Προθεματική αύξηση/μείωση, συμπλήρωμα, λογικό NOT	++, --, ~, !
Αριθμητικοί	Πολλαπλασιασμός	*, /, %
	Πρόσθεση	+, -
Διολίσθησης	Διολίσθηση	<<, >>, >>>
Σχεσιακοί	Σύγκριση	<, >, <=, >=, instanceof
	Έλεγχος ισότητας	==, !=
Ψηφίου	AND ψηφίο προς ψηφίο	&
	XOR ψηφίο προς ψηφίο	^
	OR ψηφίο προς ψηφίο	|
Λογικοί	Λογικό AND	&&
	Λογικό OR	||
Τριαδικός		? :

Εκχώρησης	Εκχώρησης	=, +=, -=, *=, /=, %=, &=, ^=, |=, <<=, >>=, >>>=

***Πίνακας 4.1**: Προτεραιότητα τελεστών*

Ας υποθέσουμε την αλγεβρική έκφραση x=5+3-2. Η πρόσθεση και η αφαίρεση έχουν ίδια προτεραιότητα μεταξύ τους και ψηλότερη από την εκχώρηση. Επομένως, πρώτα θα υπολογιστεί η έκφραση 5+3-2 και το αποτέλεσμά της θα εκχωρηθεί στη μεταβλητή x. Πώς όμως θα αξιολογηθεί το τμήμα 5+3-2; Ο κανόνας εδώ είναι πως η αξιολόγηση γίνεται από αριστερά προς τα δεξιά, δηλαδή γίνεται πρώτα η πρόσθεση και το αποτέλεσμά της συνιστά τελεστέο για την αφαίρεση. Το ίδιο ισχύει για τις περισσότερες εκφράσεις της Java. Οι υπολογισμοί γίνονται από αριστερά προς τα δεξιά για όλους τους τελεστές πλην της εκχώρησης όπου η αξιολόγηση γίνεται από δεξιά προς τα αριστερά.

4.1.1 Ο τελεστής εκχώρησης

Η εκχώρηση είναι δυαδική πράξη και ο βασικός τελεστής της συμβολίζεται με τον χαρακτήρα '='. Στη γραμμή 2 του κώδικα 4.1 δηλώνονται τρεις ακέραιες μεταβλητές και αρχικοποιούνται οι δύο από αυτές. Στη γραμμή 3, εκχωρείται η τιμή 8 στη μεταβλητή i. Συνηθίζεται η πρώτη εκχώρηση σε μεταβλητή να αποκαλείται αρχικοποίηση (initialization). Η έκφραση (expression) στη γραμμή 3, επομένως, μπορούμε να πούμε πως αρχικοποιεί την i. Η αξιολόγηση γίνεται από τη δεξιά προς την αριστερή πλευρά της έκφρασης. Αυτό σημαίνει πως πρώτα θα υπολογιστεί ότι υπάρχει στα δεξιά του τελεστή εκχώρησης και στη συνέχεια η τιμή που θα προκύψει θα εκχωρηθεί στην i. Βέβαια, στη συγκεκριμένη περίπτωση, δεν απαιτούνται ιδιαίτεροι υπολογισμοί δεδομένου ότι στα δεξιά του τελεστή υπάρχει μια αριθμητική σταθερά. Θα μπορούσε όμως να υπάρχει και μια έκφραση που το αποτέλεσμα της αξιολόγησής της θα ήταν ακέραιος. Στη συνέχεια αυτής της ενότητας θα δούμε αρκετά τέτοια παραδείγματα.

```
1      static void assignmentDemo() { //Κώδικας 4.1
2          int i, j = 5, k = 9;
3          i = 8;
4          System.out.println(j = i);
5          j = k = i;
6          System.out.println(i + " " + j + " " + k);
7      }
```

***Κώδικας 4.1**: Ο τελεστής εκχώρησης*

Εκείνο όμως που έχει ιδιαίτερο ενδιαφέρον είναι πως η εκχώρηση επιστρέφει στο περιβάλλον κλήσης της μια τιμή και συγκεκριμένα την τιμή της i μετά την εκχώρηση. Αυτή η τιμή δεν αξιοποιείται στη γραμμή 3. Αξιοποιείται όμως στη γραμμή 4 που τυπώνει την έκφραση j=i. Αν τρέξετε τον κώδικα 4.1, θα διαπιστώσετε πως η τιμή που τυπώνεται είναι 8. Η εκχώρηση, αφού αξιολόγησε την τιμή του i, την εκχώρησε στην j και επέστρεψε στην println την τιμή του j μετά την εκχώρηση. Σε αυτό το χαρακτηριστικό βασίζεται και η υποστήριξη της πολλαπλής εκχώρησης (γραμμή 5).

Όπως φαίνεται στον πίνακα 4.1, είναι διαθέσιμοι και άλλοι τελεστές εκχώρησης. Η παρουσίασή τους γίνεται πιο κάτω σε αυτήν την ενότητα.

4.1.2 Αριθμητικοί τελεστές

Η Java παρέχει τους δυαδικούς τελεστές πρόσθεση, αφαίρεση, πολλαπλασιασμό, διαίρεση, υπόλοιπο διαίρεσης. Όλοι αυτοί οι τελεστές είναι δυαδικοί.

4.1.2.1 Πρόσθεση

Ο τελεστής πρόσθεσης συμβολίζεται με τον χαρακτήρα '+'. Στον κώδικα 4.2, προσθέτουμε το i με το j. Το αποτέλεσμα στέλνεται στην έξοδο όπου και τυπώνεται η τιμή 5. Στη γραμμή 4 όμως, το αποτέλεσμα ίσως φανεί λίγο παράδοξο. Πράγματι, η έξοδος του κώδικα της γραμμής 4 είναι i+j=32. Ο χαρακτήρας '+' χρησιμοποιείται τόσο για την πρόσθεση μεταξύ αριθμών, όσο και για τη σύνδεση String (concatenation)

μεταξύ τους. Το αν θα ληφθεί ως πρόσθεση αριθμών ή ως σύνδεση String εξαρτάται από τους τελεστέους που συντάσσονται μαζί του. Αν έστω και ένας από τους τελεστέους είναι String, τότε εκτελείται η σύνδεση Strings. Στη γραμμή 4, η αξιολόγηση γίνεται από αριστερά προς τα δεξιά. Έτσι, στην έκφραση "i+j="+i εντοπίζεται πρώτα το String "i+j=", οπότε λαμβάνεται η απόφαση πως πρόκειται για σύνδεση. Έτσι ο επόμενος ακέραιος τελεστέος μετατρέπεται αυτόματα σε String, έτσι το αποτέλεσμα "i+j="+i είναι "i+j=3" και ο τύπος του είναι String. Στη συνέχεια μένει να συνδεθεί αυτό το String με την ακέραιη μεταβλητή j. Με παρόμοια διαδικασία, το τελικό αποτέλεσμα είναι "i+j=32".

```
1     static void additionDemo() { //Κώδικας 4.2
2         int i = 3, j = 2;
3         System.out.println(i + j);
4         System.out.println("i+j= " + i + j); //concatenation
5         System.out.println("i+j= " + (i + j));
6         String h = "hello", w = "World";
7         System.out.println(h + " " + w);
8         double k = 1;
9         float m = 1f;
10        k = k + m;
11        char a = 'A';
12        System.out.println(a + 1);
13        System.out.println((char) (a + 1));
14    }
```

***Κώδικας 4.2**: Ο τελεστής πρόσθεσης*

Αν εκείνο που θέλουμε να πετύχουμε είναι να προστεθούν πρώτα το i με το j και το αποτέλεσμα να συνδεθεί με το "i+j= " ώστε να εκτυπωθούν μαζί, τότε θα πρέπει να δώσουμε προτεραιότητα στην πρόσθεση των ακέραιων, όπως δείχνει η γραμμή 5. Ισχύει και εδώ, όπως και στην άλγεβρα πως οι εκφράσεις μέσα σε παρενθέσεις αξιολογούνται κατά προτεραιότητα. Στη συνέχεια, οι γραμμές 22 και 23 παρουσιάζουν μια περίπτωση σύνδεσης String. Οι γραμμές, 8, 9 και 10 επιδεικνύουν πως ο τελεστής πρόσθεσης συντάσσεται με όλους τους αριθμητικούς τύπους. Ωστόσο, θα πρέπει να έχουμε υπόψη πως όταν συντάσσεται με δύο διαφορετικούς αριθμητικούς τύπους, ο τύπος του αποτελέσματος αποκτάται από τον τελεστέο που έχει το μεγαλύτερο μέγεθος. Στο παράδειγμά μας, η έκφραση k+m επιστρέφει double. Αν επιχειρούσαμε να εκχωρήσουμε αυτό το αποτέλεσμα σε μεταβλητή τύπου float, ο μεταγλωττιστής θα παρήγαγε λάθος. Όπως έχουμε αναφέρει στην ενότητα 3.2, το αποτέλεσμα των εκφράσεων με byte ή short είναι αυτόματα int.
Ο τελεστής της πρόσθεσης δέχεται και τελεστέους τύπου char. Στην περίπτωση αυτή προστίθεται ο κωδικός των χαρακτήρων και το αποτέλεσμα της πράξης είναι τύπου int. Έτσι η γραμμή 12 θα προσθέσει στο 65 που είναι ο κωδικός του 'A' το 1 και θα τυπώσει το 66 που είναι ο κωδικός του 'B'. Μπορούμε να δούμε τον χαρακτήρα 'B' αν μετατρέψουμε τον τύπο του αποτελέσματος (type casting) όπως δείχνει η γραμμή 13.

4.1.2.2 Αφαίρεση

Ο τελεστής αφαίρεσης συμβολίζεται με τον χαρακτήρα '-'. Η γραμμή 3 του κώδικα 4.3 παράγει λάθος μεταγλώττισης εφόσον αποσχολιαστεί. Πράγματι, ανάλογα με όσα είπαμε στην περιγραφή του τελεστή πρόσθεσης, η έκφραση "i-j="+i επιστρέφει String. Στη συνέχεια η αφαίρεση ενός ακεραίου από ένα String δεν ορίζεται. Επομένως, σε αυτήν την περίπτωση, είμαστε αναγκασμένοι να δώσουμε προτεραιότητα στην αφαίρεση όπως φαίνεται στη γραμμή 4. Η αφαίρεση συντάσσεται και με δεδομένα τύπου char. Η συμπεριφορά της σε αυτήν την περίπτωση είναι ανάλογη με την πρόσθεση. Επίσης, ανάλογη με την πρόσθεση συμπεριφορά παρουσιάζει η αφαίρεση στην περίπτωση τελεστέων διαφορετικών αριθμητικών τύπων.

```
1     static void subtractionDemo() { //Κώδικας 4.3
2         int i = 3, j = 2;
3         //System.out.println("i-j= "+i-j);
```

```
4         System.out.println("i-j= " + (i - j));
5         char bita = 'B';
6         System.out.println(bita - 1);
7     }
```

***Κώδικας 4.3**: Ο τελεστής αφαίρεσης*

4.1.2.3 Πολλαπλασιασμός

Ο τελεστής του πολλαπλασιασμού συμβολίζεται με τον χαρακτήρα αστερίσκο, '*'. Το αποτέλεσμα της γραμμής 4 του κώδικα 4.4 είναι το αναμενόμενο. Αυτό οφείλεται στο γεγονός ότι ο πολλαπλασιασμός έχει προτεραιότητα έναντι της πρόσθεσης. Επομένως, στη γραμμή 4 υπολογίζεται πρώτα το γινόμενο i*j και στη συνέχεια συνδέεται το String "i*j=" με το αποτέλεσμα του πολλαπλασιασμού. Κατά συνέπεια οι παρενθέσεις που περικλείουν το γινόμενο i*j, στη γραμμή 5, είναι περιττές χωρίς να οδηγούν σε λάθος. Ο πολλαπλασιασμός δέχεται και τελεστέους τύπου char. Το αποτέλεσμα είναι το γινόμενο των κωδικών των χαρακτήρων. Έτσι, η γραμμή 6 θα τυπώσει 132, δηλαδή τον κωδικό του B επί 2.

```
1     static void multiplicationDemo() { //Κώδικας 4.4
2        int i = 3, j = 2;
3        char bita = 'B';
4        System.out.println("i*j= " + i * j);
5        System.out.println("i*j= " + (i * j));
6        System.out.println("bita*2=" + bita * 2);
7     }
```

***Κώδικας 4.4**: Ο τελεστής του πολλαπλασιασμού*

4.1.2.4 Διαίρεση

Ο τελεστής της διαίρεσης συμβολίζεται με τον χαρακτήρα πλάγια κάθετος, '/'. Ό,τι αναφέρθηκε για τον πολλαπλασιασμό, ισχύει κατ' αντιστοιχία και για τη διαίρεση. Επιπλέον, η διαίρεση διακρίνεται σε διαίρεση μεταξύ πραγματικών και διαίρεση μεταξύ ακέραιων. Αν οι τελεστέοι είναι ακέραιοι, τότε έχουμε ακέραια διαίρεση. Αν ένας τουλάχιστον από τους τελεστέους είναι πραγματικός, τότε έχουμε διαίρεση.

```
1     static void divisionDemo() { //Κώδικας 4.5
2        int i = 3;
3        int j = 2;
4        System.out.println("i/j= " + i / j);
5        System.out.println("i/j= " + (i / j));
6        System.out.println();
7        double d = 3, b = 2;
8        System.out.println("d/b= " + (d / b));
9        System.out.println("(double)i/j= " + (double) i / j);
10    }
```

***Κώδικας 4.5**: Ο τελεστής της διαίρεσης*

Η έξοδος της γραμμής 4 του κώδικα 4.5 είναι i/j=1. Και οι δύο τελεστέοι είναι ακέραιοι, οπότε η Java εκτελεί διαίρεση ακέραιων. Αντίθετα στη γραμμή 8, η έξοδος είναι d/b=1.5. Αν επιθυμούμε να κάνουμε διαίρεση πραγματικών μεταξύ ακέραιων μεταβλητών, θα πρέπει να μετατρέψουμε τον τύπο τουλάχιστον σε έναν από τους τελεστέους όπως φαίνεται στη γραμμή 9.

4.1.2.5 Υπόλοιπο διαίρεσης

Το υπόλοιπο συμβολίζεται με τον χαρακτήρα επί τοις εκατό, '%'. Εδώ έχουμε διαφοροποίηση μεταξύ υπολοίπου που προκύπτει από διαίρεση πραγματικών και υπολοίπου διαίρεσης ακέραιων.

```
1     static void moduloDemo() { //Κώδικας 4.6
2        int i = 3, j = 2; //1
```

```
3         System.out.println("i%j= " + i % j);
4         System.out.println("i%j= " + (i % j));
5         double d = 7.5, b = 3.5;
6         System.out.println("d%b= " + d % b);
7         System.out.println("d%b= " + (d % b));
8     }
```

***Κώδικας 4.6**: Το υπόλοιπο διαίρεσης*

Η έξοδος της γραμμής 3 του κώδικα 4.6 είναι i%j=1. Ο τύπος της έκφρασης i%j είναι int. Στη γραμμή 6 εκτελείται διαίρεση μεταξύ πραγματικών και η έξοδος είναι d%b=0.5. Ο τύπος της έκφρασης d%b είναι double.

4.1.3 Μοναδιαίοι τελεστές

Οι μοναδιαίοι τελεστές που υποστηρίζονται είναι το θετικό πρόσημο, το αρνητικό πρόσημο, οι τελεστές προσαύξησης, οι τελεστές μείωσης και ο λογικός τελεστής αντιστροφής.

4.1.3.1 Οι τελεστές προσήμου

Ο μοναδιαίος τελεστής μείον (unary minus operator) συμβολίζεται με τον χαρακτήρα '-' και επιστρέφει τον αντίθετο του τελεστέου. Στη γραμμή 5, ο κώδικας 4.7, τυπώνει -1. Ο τελεστής μείον δέχεται ως τελεστέους και πραγματικούς αλλά και χαρακτήρες. Ανάλογη είναι η λειτουργία του μοναδιαίου τελεστή συν (unary plus operator), '+'. Ωστόσο, ο τελεστής συν, ισχύει εξ ορισμού και έτσι δεν αναφέρεται σχεδόν ποτέ σε κώδικες Java.

```
1     static void unaryOps() { //Κώδικας 4.7
2         int i = 1;
3         System.out.println(i);
4         System.out.println(+i);
5         System.out.println(-i);
6         System.out.println();
7
8         System.out.println("Postfix increment");
9         System.out.println(i++);
10        System.out.println(i);
11        System.out.println();
12
13         System.out.println("Prefix increment");
14         i = 1;
15         System.out.println(++i);
16         System.out.println(i);
17         System.out.println();
18
19         System.out.println("Postfix decrement");
20         i = 1;
21         System.out.println(i--);
22         System.out.println(i);
23         System.out.println();
24
25         System.out.println("Prefix decrement");
26         i = 1;
27         System.out.println(--i);
28         System.out.println(i);
29         System.out.println();
30
31         boolean b = false;
32         System.out.println("b= " + b + ", !b= " + !b);
```

```
33      }
```

Κώδικας 4.7*: Μοναδιαίοι τελεστές*

4.1.3.2 Τελεστές προσαύξησης

Ο τελεστής προσαύξησης συμβολίζεται με τους χαρακτήρες "++" και παρέχεται σε δύο μορφές, στον προθεματικό τελεστή προσαύξησης (prefix increment operator) και στον μεταθεματικό τελεστή προσαύξησης (postfix increment operator). Όταν ο τελεστής προηγείται του τελεστέου, έχουμε τον προθεματικό τελεστή. Αντίθετα, έχουμε τον μεταθεματικό όταν ο τελεστέος προηγείται του τελεστή. Η λειτουργία των δύο τελεστών διαφέρει. Ο προθεματικός τελεστής αυξάνει κατά 1 την τιμή του τελεστέου και επιστρέφει την αυξημένη τιμή. Αντίθετα, ο μεταθεματικός τελεστής αυξάνει την τιμή του τελεστέου κατά 1, επιστρέφει όμως την τιμή που είχε ο τελεστέος πριν την αύξηση. Ο κώδικας στη γραμμή 9 θα τυπώσει 1. Ο μεταθεματικός τελεστής αύξησε την τιμή του i κατά 1 αλλά επέστρεψε την τιμή που είχε το i πριν την αύξηση. Στη συνέχεια στη γραμμή 10 θα τυπωθεί η αυξημένη τιμή, δηλαδή το 2. Αντίθετα, στη γραμμή 15 θα τυπωθεί 2. Επίσης, 2 θα τυπωθεί στη γραμμή 16.

4.1.3.3 Τελεστές μείωσης

Παρόμοια είναι η λειτουργία των τελεστών μείωσης. Ο μεταθεματικός τελεστής μείωσης μειώνει την τιμή του τελεστέου κατά 1, επιστρέφει όμως την τιμή πριν τη μείωση. Αντίθετα, ο προθεματικός τελεστής μείωσης μειώνει την τιμή του τελεστέου κατά 1 και επιστρέφει την τιμή μετά τη μείωση.
Στη γραμμή 21 του κώδικα 4.7, θα τυπωθεί 1 ενώ στη γραμμή 22 θα τυπωθεί 0. Αντίθετα, στις γραμμές 27 και 28 θα τυπωθεί 0.

4.1.3.4 Ο λογικός τελεστής αντιστροφής

Ο λογικός τελεστής αντιστροφής (logical complement operator) συμβολίζεται με τον χαρακτήρα '!' και συντάσσεται με τύπο boolean. Το αποτέλεσμα του τελεστή είναι η αντιστροφή της τιμής του τελεστέου. Έτσι, ο κώδικας στη γραμμή 32, θα τυπώσει b= false, !b= true.

4.1.4 Σχεσιακοί τελεστές

Υποστηρίζονται οι ακόλουθοι σχεσιακοί τελεστές: ο τελεστής ελέγχου ισότητας, ο τελεστής ελέγχου ανισότητας, ο τελεστής μεγαλύτερο από, ο τελεστής μεγαλύτερο ή ίσο, ο τελεστής μικρότερο από και ο τελεστής μικρότερο ή ίσο. Όλοι οι σχεσιακοί τελεστές είναι δυαδικοί και επιστρέφουν boolean τιμή.

4.1.4.1 Ο τελεστής ελέγχου ισότητας

Ο τελεστής ελέγχου ισότητας (equality operator) συμβολίζεται με "==" και επιστρέφει true αν και μόνο αν οι τελεστέοι του έχουν την ίδια τιμή. Στην περίπτωση που οι τελεστέοι είναι ακέραιοι τύποι ή boolean, η λειτουργία του τελεστή είναι απλή. Η γραμμή 3 του κώδικα 4.8, θα τυπώσει i==j is true.

Στην περίπτωση όμως πραγματικών τελεστέων δεν ισχύει το ίδιο. Στη γραμμή 4, ορίζουμε τρεις μεταβλητές τύπου double. Η μεταβλητή d αρχικοποιείται στην τιμή 1+2*k. Στη συνέχεια, στις γραμμές 5 και 6 προσθέτουμε στην b την k, δηλαδή προσθέτουμε δύο φορές το k. Σύμφωνα με τα μαθηματικά αναμένουμε πως το d είναι ίσο με το b. Ωστόσο, λόγω του προσεγγιστικού χαρακτήρα και της διαφορετικής σειράς που γίνονται οι πράξεις κατά τον υπολογισμό των d και b, το d δεν είναι ίσο με το b. Η έξοδος της γραμμής 7 είναι b=1.0000200000000001 and d=1.00002. Λογική συνέπεια είναι πως η γραμμή 8 τυπώνει false. Ο γενικός κανόνας είναι πως η χρήση του τελεστή ισότητας με πραγματικούς τελεστέους πρέπει να αποφεύγεται. Αντίθετα, για τη σύγκριση πραγματικών αριθμών θα πρέπει να χρησιμοποιείται κατάλληλη συνάρτηση προσεγγιστικής ισότητας που παρουσιάστηκε στην ενότητα 3.2.

Η συμπεριφορά του τελεστή με τύπους αναφοράς φαίνεται διαφορετική καθώς στην περίπτωση αυτή συγκρίνονται διευθύνσεις και όχι περιεχόμενα. Οι αναφορές τύπου String s1 και s2 δείχνουν στο ίδιο String στη μνήμη το περιεχόμενο του οποίου είναι "John". Επομένως η γραμμή 11 τυπώνει s1==s2 is true. Αντίθετα, η s3 αναφέρεται σε ένα String, που, παρότι το περιεχόμενό του είναι "John", βρίσκεται σε άλλη θέση της μνήμης. Επομένως, η γραμμή 12 τυπώνει s1==s3 is false. Αν θέλουμε να συγκρίνουμε για ισότητα δύο Strings ως προς το περιεχόμενό τους, μπορούμε να χρησιμοποιήσουμε τη συνάρτηση equals όπως φαίνεται στη γραμμή 13. Η έξοδος της γραμμής 13 είναι s1.equals(s3) is true. Κατά αντίστοιχο τρόπο

λειτουργεί ο τελεστής ισότητας για όλους τους μη θεμελιώδεις τύπους. Κάθε μεταβλητή αναφοράς που έχει λάβει την τιμή της από την επιστροφή της new δεν μπορεί να συγκριθεί με τον τελεστή ισότητας αλλά με τη συνάρτηση equals.

```
1      static void equalityOp() { //Κώδικας 4.8
2          int i = 1, j = 1;
3          System.out.println("i==j is " + (i == j));
4          double k = 0.00001, d = 1 + 2 * k, b = 1;
5          b = b + k;
6          b = b + k;
7          System.out.println("b=" + b + " and d=" + d);
8          System.out.println(b == d);
9          System.out.println(Double.compare(b,d));
10         String s1 = "John", s2 = "John", s3 = new String("John");
11         System.out.println("s1==s2 is " + s1 == s2);
12         System.out.println("s1==s3 is " + s1 == s3);
13         System.out.println("s1.equals(s3) is " + s1.equals(s3));
14     }
```

***Κώδικας 4.8**: Ο τελεστής ελέγχου ισότητας*

4.1.4.2 Ο τελεστής ανισότητας

Ο τελεστής ανισότητας (inequality operator) συμβολίζεται με "!=". Ο τελεστής λειτουργεί με ανάλογο τρόπο και περιορισμούς με τον τελεστή ισότητας. Ισχύει !(a==b) == (a!=b), δηλαδή αν αντιστρέψουμε τον έλεγχο ισότητας μεταξύ δύο τελεστέων θα λάβουμε το ίδιο αποτέλεσμα με αυτό που θα λάβουμε εφαρμόζοντας τον τελεστή ανισότητας στους ίδιους τελεστέους. Επομένως, οι παρατηρήσεις σχετικά με τους πραγματικούς αριθμούς και τις μεταβλητές αναφοράς που επισημάνθηκαν στον τελεστή ισότητας ισχύουν κατ' αναλογία και εδώ.

4.1.4.3 Οι υπόλοιποι σχεσιακοί τελεστές

Πρόκειται για τον τελεστή μεγαλύτερο από (greater than) που συμβολίζεται με τον χαρακτήρα '>', τον τελεστή μεγαλύτερο ή ίσο από (greater than or equal to) που συμβολίζεται με τον χαρακτήρα '>=', τον τελεστή μικρότερο από (less than) που συμβολίζεται με τον χαρακτήρα '<' τον τελεστή μικρότερο ή ίσο από (less than or equal to) που συμβολίζεται με τον χαρακτήρα '<='. Οι τελεστές αυτοί δέχονται ως τελεστέους όλους τους θεμελιώδεις τύπους πλην του τύπου boolean. Δεν συντάσσονται με μη θεμελιώδεις τύπους. Με πραγματικούς τελεστέους, θα πρέπει να χρησιμοποιούνται με την επιφύλαξη που προκύπτει από τον προσεγγιστικό χαρακτήρα των πραγματικών.

```
1      static void relationalOps() { //Κώδικας 4.9
2          int i = 5, j = 2;
3          char a = 'A', b = 'B';
4          System.out.println(i > j);
5          System.out.println(b > a);
6      }
```

***Κώδικας 4.9**: Ο τελεστής μεγαλύτερο από*

Η έξοδος της γραμμής 4 του κώδικα 4.9 εμφανίζει true. Στη γραμμή 5 συγκρίνονται δύο χαρακτήρες. Η έξοδος της γραμμής είναι επίσης true, καθώς εκείνο που συμβαίνει είναι πως συγκρίνονται οι κωδικοί των χαρακτήρων. Παρότι οι τελεστές αυτοί δεν συντάσσονται με String, αξίζει να σημειωθεί πως γενικότερα η σύγκριση μεταξύ αριθμών είναι αριθμητική, ενώ η σύγκριση μεταξύ σειρών χαρακτήρων είναι λεξικογραφική. Σύμφωνα με την αριθμητική σύγκριση, το 10 είναι μεγαλύτερο από το 2, ενώ σύμφωνα με τη λεξικογραφική το "10" είναι μικρότερο από το "2".

4.1.5 Λογικοί τελεστές

Περιλαμβάνονται οι λογικοί τελεστές and και or, ο τριαδικός τελεστής και ο τελεστής instanceof. Ο τελεστής instanceof σχετίζεται με το αντικειμενοστρεφές μοντέλο και η παρουσίασή του γίνεται στην ενότητα 13.2.1.

4.1.5.1 Λογικό and

Το λογικό and συμβολίζεται ως "&&", είναι δυαδικός τελεστής και συντάσσεται αποκλειστικά με τελεστέους τύπου boolean. Η τιμή επιστροφής του υπολογίζεται όπως δείχνει ο πίνακας 4.2.

Τελεστέος 1	**Τελεστέος 2**	**Αποτέλεσμα**
TRUE	TRUE	TRUE
TRUE	FALSE	FALSE
FALSE	TRUE	FALSE
FALSE	FALSE	FALSE

Πίνακας 4.2 *Πίνακας αληθείας λογικού and*

Η υλοποίηση του λογικού and ακολουθεί τη λογική βραχέως κυκλώματος (short circuit). Η αξιολόγηση γίνεται από αριστερά προς τα δεξιά. Αν ο πρώτος τελεστέος αρκεί για τον υπολογισμό του αποτελέσματος, ο δεύτερος τελεστέος δεν αξιολογείται. Σύμφωνα με τον πίνακα 2, όταν ο τελεστέος 1, έχει την τιμή false, τότε το αποτέλεσμα της έκφρασης είναι false ανεξάρτητα από την τιμή του δεύτερου τελεστέου. Αντίθετα, όταν ο πρώτος τελεστέος έχει την τιμή true, τότε το αποτέλεσμα της έκφρασης εξαρτάται από την τιμή του δεύτερου τελεστέου.

```
1    static void shortCircuitAnd() { //Κώδικας 4.10
2        int i = 1, j = 2;
3        System.out.println("i++>1 && j--<3 is " + (i++ > 1 && j-- < 3));
4        System.out.println("i= " + i + " j= " + j);
5        i = 1;
6        j = 2;
7        System.out.println("++i>1 && j--<3 is " + (++i > 1 && j-- < 3));
8        System.out.println("i= " + i + " j= " + j);
9    }
```

Κώδικας 4.10*: Short circuit and*

Η γραμμή 3 του κώδικα 4.10 τυπώνει i++>1 && j--<3 is false. Ο πρώτος τελεστέος είναι η λογική έκφραση i++>1, η τιμή του είναι false. Επομένως, ο δεύτερος, δηλαδή το j--<3 δεν θα αξιολογηθεί και άρα δεν θα μειωθεί η τιμή του j με αποτέλεσμα η έξοδος της γραμμής 4 είναι i= 2 j= 2. Αντίθετα, στη γραμμή 7, ο πρώτος τελεστέος του λογικού and είναι ++i>1 και η τιμή του είναι true. Επομένως θα αξιολογηθεί και ο δεύτερος τελεστέος με αποτέλεσμα να μειωθεί το j. Καθώς και ο δεύτερος τελεστέος είναι true, το αποτέλεσμα του λογικού and είναι επίσης true. Επομένως, η γραμμή 7 θα τυπώσει ++i>1 && j--<3 is true και η γραμμή 8 θα τυπώσει i= 2 j= 1.

4.1.5.2 Λογικό or

Το λογικό or συμβολίζεται ως "||", είναι δυαδικός τελεστής και συντάσσεται αποκλειστικά με τελεστέους τύπου boolean. Η τιμή επιστροφής του υπολογίζεται όπως δείχνει ο πίνακας 4.3.

Τελεστέος 1	**Τελεστέος 2**	**Αποτέλεσμα**
TRUE	TRUE	TRUE
TRUE	FALSE	TRUE